

动态规划

grass8cow

2026.1.19

qoj8528 Chords

有一个环， $2n$ 个点间连了 n 条边，每个点恰好是一条边的端点。

要取出尽量多的边使它们两两不交，数据随机。

$$n \leq 10^5$$

qoj8528 Chords

先编出来 $O(n^2)$ 的 dp，就是设 $dp_{i,j}$ 为 $[i,j]$ 内能取多少线段。

转移分类讨论，首先用 $\max(dp_{i-1,j}, dp_{i,j-1})$ 更新 $dp_{i,j}$ ，漏掉的情况只有： i,j 间有边。

就用 $dp_{i+1,j-1} + 1$ 更新 $dp_{i,j}$ 即可。

怎么优化呢？tips：随机情况下答案极大概率是 $O(\sqrt{n})$ 级别的

qoj8528 Chords

交换值域和状态。

考虑这样设状态：令 $pr_{l,j}$ 为最小的 r 使得 $dp_{l,r} \geq j$ 。

首先可以用 $pr_{l+1,j}$ 来更新 $pr_{l,j}$ 。然后就是讨论取不取 l 所在的线段。

设 l 所在线段的右端为 p ， $T = dp_{l+1,p-1}$ 。就可以用 $pr_{p+1,j-T-1}$ 更新 $pr_{l,j}$ 。

复杂度 $O(n\sqrt{n})$ 。

gym104053D Digits

给定一个整数序列 a ，现在我们要把它划分成若干连续段。定义一个划分方案是好的，当且仅当：

把每个连续段的 sum 计算出来，并当作一个字符串。把这些连续段得到的字符串拼起来之后，得到的是回文串。

举个例子： $a = \{5, 7, 10, 11\}$ 。一个划分方案是在中间劈一刀，那每个连续段相加就得到了 12, 21，拼起来之后 1221 是回文串。

多测， $\sum n \leq 200$ ，答案对 $10^9 + 9$ 取模。时限 2s

gym104053D Digits

为了处理回文，可以考虑这样划分阶段：扩展前缀和后缀的划分，并从两端消去相同的字符。

考虑设 $f_{i,j,s}$ 表示：有多少个划分 $[i, j]$ 的方案，使得这个方案得到的字符串在后面拼上 s 后是回文的。 $g_{i,j,s}$ 就是 s 接在前面的方案数。

转移以 $f_{i,j,s}$ 为例，我们枚举划分出的第一段 $[i, k]$ ，让 $[i, k]$ 得到的串 t 和 s 进行消除，如果两个串都没有消完就寄了；否则根据哪个串非空，用 $f_{k+1,j,s'}$ 或者 $g_{k+1,j,t'}$ 转移即可。

gym104053D Digits

分析一下这样做的复杂度，总状态数是 $n^3 \log V$ 种的：还是以 $f_{i,j,s}$ 为例， s 一定是某个 $[j+1, k]$ 形成的串的某个前缀。于是复杂度为 $O(n^4 \log^2 V)$ 。

看似不能通过，但其实状态卡不满，利用 unordered map 是能轻松通过的。

gym104053D Digits

还是想想怎么优化吧。还是以计算 f 为例，我们把转移分成两种： s' 为空， t' 为空。

对于 t' 为空的情况，相当于 t 是 s 某个后缀 reverse 后的结果。

计算 $f'_{i,j,s}$ 表示 $\sum_{k=i}^{j+1} f_{k,j,s}$ ，然后枚举可能的 t ，发现我们需要的 k 构成一个区间 $[l, r]$ ，于是用 $f'_{l,j,s'} - f'_{r+1,j,s'}$ 即可计算。

对于 s' 为空的情况，我们不要直接枚举 s 。先枚举 i, j, k ，把 t 先计算出来。那么能被转移的 s 只能是 t 一个前缀 reverse 的结果。

这个时候再枚举可能的 s ，用 $g_{k+1,j,t'}$ 转移给 $f_{i,j,s}$ 即可。

复杂度 $O(n^3 \log V (\log n + \log V))$ 。

P10104 异或图

给定一张 n 个点 m 条边的无向图和一个长度为 n 的数组 $a_1, a_2, \dots, a_n$ 以及一个整数 C ，你需要求出有多少个长度为 n 的数组 b 满足：

1. $0 \leq b_i \leq a_i, \forall 1 \leq i \leq n$ 。
2. 对于每条边 (u, v) , $b_u \neq b_v$ 。
3. $b_1 \oplus b_2 \oplus \dots \oplus b_n = C$ ，其中 $\oplus$ 代表异或。

答案对 998244353 取模。

$$1 \leq n \leq 15$$

P10104 异或图

互不相等容斥（有些人称为集合划分容斥）。也就是枚举一个边集，强制这些边的两个端点 b_i 相同。

那我们先思考，如果确定了这个边集，怎么算答案。

能转化成以下问题：给定集合 $S \subseteq [n]$ ， $\forall x \in S$ 都有 $0 \leq b_x \leq a_x$ ，并要求 $\oplus_{x \in S} b_x = C$ ，求方案数。

这可以数位 dp 解决。

我们称这个集合是“生效集合”，发现难点在于：怎么对于 $[n]$ 的每个子集 S ，计算生效集合是 S 的边集的容斥系数之和。

P10104 异或图

先把这些数按 a 排序，然后考虑 $f(S, T)$ 表示当前已经确定了 S 内的容斥方案，其中生效集合为 T ，直接转移是 $O(4^n)$ 的，但你发现记忆化搜索一下就过了，why?

我们转移时为了计数不重，枚举的下一个联通块会包含第一个不在 S 里的元素。而且新增的“生效元素”只会是这个元素。

那么 T 不仅是 S 的子集，而且取出 S 的第一个 0 后， T 中的 1 都在这个 0 前面。也就是可以改写成 $f(S', i)$ 表示 S 中第一个 0 在 i ， i 后 S 的状态和 i 前 T 的状态拼起来的结果。这样状态就是 $O(2^n n)$ 的，总复杂度 $O(3^n n)$ 。

P10104 异或图

你发现我们其实还漏掉了一个东西：怎么对所有 $S \subseteq [n]$ 计算 S 的导出子图中所有联通子图的 $(-1)^{|E|}$ 之和。
这可以用我们明天将要讨论的“集合幂级数”做到 $O(2^n n^2)$ 。

P9732 Trade

有 n 个机器人排成一排，第 i 个机器人的购买价是 c_i 欧元，卖出价是 s_i 欧元。

给定 $1 \leq k \leq n$ ，你需要购买一段长度至少为 k 的区间中所有的机器人，然后选择其中的恰好 k 个机器人来卖出。

你需要求出：1. 你能够得到的最大收益；2. 在收益最大化的前提下，哪些机器人可以在某种最优方案中被卖出。

$1 \leq k \leq n \leq 250000$, $1 \leq c_i, s_i \leq 10^9$ 。时限 7s

先想想怎么算第一问。我们设 $f(l, r)$ 是前 k 大 a 的和减掉 b 的和，我们要找最大的 $f(l, r)$ ，不难发现 f 是满足四边形不等式的，那我们分治一下就算出来答案了。

接下来看怎么判一个位置在不在最优方案内，只需要找出所有 $f(l, r) = ans$ 的 $[l, r]$ ，算出区间内第 k 大值 z ，那 $[l, r]$ 内那些 $a_i \geq z$ 的位置就都合法了，问题变成找 $[l, r]$ 。但你发现这样的 $[l, r]$ 数量可以卡到 $O(n^2)$ ，怎么办呢。

P9732 Trade

神秘观察：考虑 $f(a, d) = f(b, c) = ans$,其中 $a < b < c < d$ 。
那由于 $f(a, c) + f(b, d) \geq f(a, d) + f(b, c)$,
 $f(a, c), f(b, d) \leq ans$, 可以发现 $f(a, c) = f(b, d) = ans$ 。而
[a, c] 的第 k 大值肯定 $\geq$ [a, d] 的第 k 大值, [b, d] 同理, 于是可以
发现我们不需要用 [a, d] 更新答案。

考虑分治的时候在值最大的前提下取 l 最大的, 设其为 p_r 。
可以发现对于 r , 设上一个能得到的最大值的是 r' , 那你只需要
扫一遍 $l \in [p_{r'}, p_r]$ 然后更新答案即可。复杂度 $O(n \log^2 n)$ 。

给定质数 p 以及 n 个二元组，其中第 i 个二元组形如 (op_i, a_i) 。

可以进行以下过程：第一步，将二元组任意排列。第二步，令变量 $x = 1$ ，遍历 $i = 1, 2, \dots, n$ 。

若 $op_i = 0$ ，则 $x := a_i$ ；否则， $x = x * a_i \bmod p$

你想知道：有多少个整数 $k \in [0, p)$ 满足：存在一个排列二元组的方案，使得最终能凑出来 k 。

$$p \leq 2 * 10^5, n \leq 10^6, \forall 1 \leq i \leq n, 0 \leq a_i < p$$

gym103428C

先判掉不存在赋值操作的情况：此时最终 x 是确定的。

否则：考虑排列后的最后一次赋值操作。记这个位置在 i 。那么最终得到的数就等于 $\prod_{j=i}^n a_j \bmod p$ 。

也就是说我们只关心两件事情：最后一个赋值操作是谁，以及哪些乘的操作在这次赋值操作后。

设计 $dp_{i,j}$ 表示考虑了前 i 个乘的操作，能不能凑出来 j 。

初始 $\forall 1 \leq i \leq n, op_i = 0$ ，令 $dp_{0,a_i} = 1$ ，表示我们要在赋值操作选恰好一个。

接下来就对乘的操作做 01 背包：若 $dp_{i-1,j} = 1$ ，则 $dp_{i,j} = dp_{i,j*a_i} \bmod p = 1$ 。

考虑怎么优化。首先要化乘为加。先判掉 x 最终变成 0 的特殊情况，接下来就只看那些 $a_i > 0$ 的操作。

求 p 的一个原根 g ，对所有 a_i 取离散对数，即令 $a_i = g^{b_i} \bmod p$ 。然后令 $p := \varphi(p) = p - 1$ 。

这样之前的转移就从乘法变成了加法：

$$dp_{i-1,j} = 1 \rightarrow dp_{i,j} = dp_{i,(j+b_i) \bmod p} = 1。$$

再把状态的第一维以滚动数组的方式去掉。现在我们的问题是：加入物品 i 之后，怎么快速找到所有 $dp_j = 1$ 且 $dp_{(j+b_i) \bmod p} = 0$ 的位置。

Lemma

满足 $dp_j = 1, dp_{(j+b_i) \bmod p} = 0$ 的 j 的个数，等于：满足 $dp_j = 0, dp_{(j+b_i) \bmod p} = 1$ 的 j 的个数。

证明：建立一个图，在 j 和 $(j + b_i) \bmod p$ 间连边，那么这个图的每个联通块都是简单环。

不难发现一个环上相邻 01 个数等于相邻 10 个数。这就证完了。

现在只需要快速找到 $dp_j \neq dp_{(j+b_i) \bmod p}$ 的位置。

把 dp 复制一份，转化成找出 $[0, p)$ 和 $[a, p + a)$ 中所有不相同的位置。

只需要设法快速算它们的 lcp，即找到第一个不相等的位置，因为是均摊的复杂度。

二分，转化成判断两个区间是否完全相同，可以哈希。但是 dp 数组在变化啊？用树状数组来处理哈希值的计算即可。

复杂度 $O(n \log^2 n)$ 。

qoj2562 Fake Plastic Trees 2

给定一棵树，点有点权。给出 K, L, R ，你希望对于 $i = 0, 1, 2, \dots, K$ 判断：是否存在割掉 i 条边的方案，使得每个联通块点权和在 $[L, R]$ 内。
 $n \leq 1000, K \leq 50$ 。

qoj2562 Fake Plastic Trees 2

暴力 dp 就是设 $dp_{x,i,j}$ 是以 x 为根的子树内割了 i 条边, x 所在联通块是否可能为 j 。

考虑优化, 我们真的需要记下所有可能的 j 吗?

想一想, 我们只需要每个联通块在 $[L, R]$ 内就行了, 那有没有方法能把一个 j 支配掉呢?

qoj2562 Fake Plastic Trees 2

考虑三个数 $x < y < z$ 。我们证明 $z - x \leq R - L + 1$ ，那 y 就能支配掉了。

设选择了 y 之后最后形成的的大小是 a ，那如果 $a - y + z \leq R$ 就替换成 z ，否则的话把 $a - y + z \geq R + 1$ 和 $x - z \geq L - R - 1$ 两个式子加起来就能得到 $a + x - y \geq L$ 了，所以就能把 y 替换成 x 。

再分析一下 j 的值域，由于已经割出的联通块大小总和在 $[iL, iR]$ ，可以得到可能的 j 的极差是 $\leq i(R - L)$ 的。

于是可以得到可能的 j 只有 $O(K)$ 种。我们重设状态：设 $f_{x,i}$ 是所有 $dp_{x,i,j} = 1$ 且未被支配的 j 构成的集合。总复杂度 $O(nK^3)$ 。

P2305 购票

给定一棵 n 个点的树，边有边权，每次可以从点 u 跳到它的某个祖先 v ，满足 $dis(u, v) \leq l_u$ 。代价为 $p_u * dis(u, v) + q_u$ 。

求从每个点最终跳到根节点的最小代价，保证有解。

$$1 \leq n \leq 2 * 10^5$$

P2305 购票

设 d_u 是 u 到根的边权和。

显然可以设 dp_u 表示 u 出发到根的最小代价。

想想怎么优化。如果这个树形如一条链的话，其实就是对一个区间 $[l, r]$ ，查询 $\min_{l \leq v \leq r} dp_v + (d_u - d_v) * p_u$ 的最小值。

线段树套凸包。对于线段树的一个节点 $[l, r]$ ，当 dp_r 算出来之后，我们就建立 (d_v, dp_v) 构成的下凸壳，每次查询在凸包上二分即可。

复杂度 $O(n \log^2 n)$ 。

P2305 购票

那树怎么做？再套一层重链剖分，做到 $O(n \log^3 n)$ 。

还可以更好。观察我们的 dp， dp_u 是不是按照 u 的 dfs 序被计算出的？

尝试找一组编号使得：如果只看比 u 的 dfs 序小的那些点，那么能给 u 产生贡献的点构成一段区间。

取出栈序即可。这里线段树一个节点就不能套凸包了。套李超线段树。复杂度 $O(n \log^2 n)$ 。

CF1175G Yet Another Partiton Problem

给定一个数组 $a_1, a_2, \dots, a_n$ ，你需要将其分成 k 个子段（每个元素恰好属于一个子段）。

一个子段 $a_l, a_{l+1}, \dots, a_r$ 的权值定义为 $(r - l + 1) \cdot \max_{l \leq i \leq r} (a_i)$ 。一

个划分的权值是所有子段权值之和。

请你找到权值最小的划分方案。

$1 \leq n \leq 2 \times 10^4$, $1 \leq k \leq \min(100, n)$

CF1175G Yet Another Partiton Problem

设 $dp_{t,i}$ 是把前 i 个位置划分成 t 段的最大权值和，转移即 $dp_{t,i} + val(i+1, j) \rightarrow dp_{t+1,j}$ 。

枚举 t ，然后令 $f_i = dp_{t,i}$ ，现在相当于对每个 j 计算 $f_i + val(i+1, j)$ 的最大值。设这个想求出的最大值是 g_j 。

令 $x_i = \max(a_{i+1}, a_{i+2}, \dots, a_j)$ ，我们知道，如果观察 x 的连续段的变化，那每次 $j \rightarrow j+1$ 时，会删除末尾若干连续段，然后加入一个连续段。

CF1175G Yet Another Partiton Problem

那么同时考虑一个连续段对 g_j 的贡献：设这个连续段形如 (l, r, z) ，那就可以计算 $A = \max_{l \leq i \leq r} f_i - z * i$ 。然后把这个连续段对 g_j 的贡献看做 $A + z * j$ 。

很好的事情： A 是只与连续段相关的！我们就可以把问题分为两个小问题了：加入一个连续段时计算新的 A ，以及维护所有连续段的 $A + z * j$ 的最大值。

第二个问题可以用李超树维护，虽然要删除直线，但因为删除的段都在末尾，所以只用支持可撤销，这是没有难度的。

第一个问题呢？看起来能线段树套凸包，但这样就是两 $\log$ 的。

直接维护每个连续段构成的凸包，发现凸包的结构与 z 无关，所以每次“弹掉若干连续段，加入一个新连续段”时，只需要把弹掉的连续段的凸包合并起来，那么可以启发式合并。

CF1175G Yet Another Partiton Problem

最终复杂度就是 $O(nk \log n)$ 。

一个更简单的做法: cdq 分治, 转化成计算 $f_i + \max(m_i, m_j) * (j - i) \rightarrow g_j$ 。讨论 m_i, m_j 的大小关系。

如果 $m_i < m_j$, 那么对于 j 而言它可以取的 i 构成了一段后缀。并且这个范围随着 j 增大, 是越来越大的。查 $f_i - m_j * i$ 的最大值, 只需要维护 (i, f_i) 构成的凸包即可。

$m_i > m_j$ 基本是对称的。最终复杂度 $O(nk \log n)$ 。

P9266 Nawiasowe podziały

给出一个长度为 n 且仅由字符 '(' 和 ')' 组成的字符串，以及一个数字 k ，这个字符串不一定是合法的括号序列。你的任务是把它分成 k 个非空段（每个字符必须恰好属于一段内），使得每段中是合法括号序列的子串个数之和最小。

$$1 \leq k \leq n \leq 10^5。$$

P9266 Nawiasowe podziały

首先设 (是 $+1$,) 是 -1 , 令 s_i 是括号权值的前缀和。

那 $(l, r]$ 合法等同于 $s_l = s_r = \min_{i=l}^r s_i$ 。

一般这种题就直接猜答案关于 k 是凸的, 然后 wqs 二分。我们只需要证明 $val(l, r)$ 满足四边形不等式, 其中 $val(l, r)$ 表示 $[l, r]$ 内的合法子串个数。

这是一定的: 事实上, 无论“合法子段”定义方式如何, $val(l, r)$ 都满足四边形不等式。

接下来的讨论都在这个二分的基础上: $f(l, r) = val(l, r) + mid$, 需要找一个划分方式使得每一段的 f 之和尽量小。

P9266 Nawiasowe podziały

我们首先尝试决策单调性 dp 。问题是怎么快速算 f ?

建出广义的小根笛卡尔树。具体的，我们这样建树：假设要对区间 $[l, r]$ 建树，我们找到它所有最小值的位置，设为 $p_1, p_2, \dots, p_m$ 。那这个节点有 $m + 1$ 个儿子，分别是 $[l, p_1 - 1], [p_1 + 1, p_2 - 1], \dots, [p_{m-1} + 1, p_m - 1], [p_m + 1, r]$ 建出的树。对每个节点 u 维护一个集合，就是 $S_u = \{p_1, p_2, \dots, p_m\}$ 。

那 $(l, r]$ 合法当且仅当 l, r 在同一个集合中。接下来我们在树上预处理一些东西， $val(l, r)$ 就能 $O(1)$ 求了，但细节很多。复杂度 $O(n \log n \log V)$ 。

P9266 Nawiasowe podziały

想进一步优化的话，考虑 wqs 二分基本是无法避免的，于是只能避免决策单调性。

（此处作一个图）

考虑直接树形 dp。想一想，我们计算根节点 u 对答案的贡献时关心什么信息？我们关心 S_u 中哪些点要划刀。对于一个儿子，我们只关心它内部的节点有没有划至少一刀。

P9266 Nawiasowe podziały

于是设 dp_u 是在 u 内砍至少一刀，能得到的最小权值。

令 t_u 是 u 子树内所有点的 $\sum \binom{|S_u|}{2}$ 之和。

计算 dp_u 的值，就先递归求出儿子的 dp 值再转移。

这里的转移是 trivial 的，最终能写成一个可以斜率优化的形式。

这样总复杂度就是 $O(n \log V)$ 的了。

LOJ3494 保镖

题意有点长，可以自己看一下。

LOJ3494 保镖

考虑把一个走动的人的时间，地点这两个信息，看做二维平面上的一个点，我们记为 (x, t) 。

那么一个 VIP 的走动可以看成是：往左上或者右上走。把坐标变换成 $(t + x, t - x)$ ，那他就只会往上走，往右走了。

那么每个 VIP 都被转化成了一条和坐标轴平行的线段。将这些线段离散化一下。

现在一次查询可以看做：从某个起点开始，能向上/向右走。然后有一个网格图，每个边有边权 w ，如果在上面走 t 的单位长度权值就会加 wt ，要得到尽量大的权值。

LOJ3494 保镖

走的策略一定是向上走若干步，到某条横线上再往右走到第一个格点，或者是先向上再向右。走到格点之后，就只会沿着网格的边走了。可以想想怎么证明。

设 $dp_{i,j}$ 是从 i 行 j 列的点出发得到的最大权值，设 X, Y 分别是这次查询的起点出发，往上走的第一行，往右的第一列。

以第一种走法为例，那就是求 $\max_{i \geq X} dp_{i,Y} + w_{i,Y}(v_Y - y)$ 。

把询问离线下来然后枚举 Y 。从 n 到 1 枚举 X 。那可以看成看做每次插入点对 $(-w_{X,Y}, dp_{X,Y} + w_{X,Y}v_Y)$ ，一次查询就可以看做给定 k ，查 $kx + y$ 的最大值。

LOJ3494 保镖

只需要设法维护这些点构成的上凸包。发现这些点横坐标并不是单调递增的。

但一个观察是：如果一个先出现的决策，它对应的 w 没有后出现的决策大，那这个决策肯定无效：我们能简单调整。

于是加入一个点时可以直接弹掉横坐标比它更大的点，然后再正常地做凸包插点。查询在凸包上二分即可。

复杂度 $O(n^2 + q \log n)$ 。

P5972 Desant

给定一个 1 到 n 的排列 $a_{1..n}$ ，它有 $2^n - 1$ 个非空子序列。
请对于 $1 \leq k \leq n$ ，找到一个长度为 k 的子序列，使得这个子序列的逆序对数量最少，并输出逆序对数量最少的子序列的数量。
 $n \leq 40$ 。

考虑状压 dp，设计 $dp_{i,S}$ 表示确定了前 i 个数中取了哪些，取的集合是 S 的最小逆序对数以及对应方案数。可以发现这是很劣的，还不如直接枚举。

考虑优化，我们真的需要记下来这个集合 S 吗？我们只需要正确计算逆序对数就好了。

令 $a_{i+1}, a_{i+2}, \dots, a_n$ 排序后得到 $b_1, b_2, \dots, b_m$ ，可以发现我们只关心：值在 $[1, b_1)$ 的数取了多少个，值在 (b_1, b_2) 中的数取了多少个.. 值在 $(b_m, n]$ 中的数取了多少个。

把这些数压成状态就好了。对于每个 i ，可以发现会达到的状态 S 是不多的。具体的，设 t_i 是 $a_j \in (b_i, b_{i+1}]$ 的 j 个数，可以发现 S 个数就是 $\prod t_i$ 。由于 $\sum t_i = n$ ， $\prod t_i$ 最大值就是 $3^{\frac{n}{3}}$ 级别的。

精细实现即可做到 $O(3^{\frac{n}{3}} n)$ ，而且跑不满。

qoj8005 Crossing the Border

有 n 个物品，每个物品有重量和权值。

给出常数 W ，现在要把物品划分成若干集合，使得每个集合重量和都 $\leq W$ 。

定义一个集合的权值是其内部所有物品权值的最大值，现在希望让所有集合权值和最小。

输出这个最小值以及达到最小值的本质不同的方案数，对 998244353 取模。

$n \leq 22$ 。

qoj8005 Crossing the Border

把物品按权值从小到大排序。

首先 $O(3^n)$ 的 dp 是容易的：令 o_S 是 S 中的物品的重量和, p_S 是 S 的最高位, w_i 是第 i 个物品的权值。

$o_T \leq W$ 且 $p_T > p_S$ 时用 $dp_S + w_{p_T}$ 更新 $dp_{S \cup T}$ 。

乍一看这个形式是没法优化的，但需要注意到 o 并不是一个平凡的数组。

qoj8005 Crossing the Border

考虑利用折半来优化这个过程，我们把状态表示成 LR 的形式， L, R 分别表示前 $\frac{n}{2}$ 个物品的选取状态，后 $\frac{n}{2}$ 个物品的选取状态。

核心思路就是：考虑在 LR' 处同时处理 LR 到 $L'R'$ 的转移，其中 $L \subseteq L', R \subseteq R'$ 。

qoj8005 Crossing the Border

具体的，我们枚举 L 和 R' ，如果 $R' = 0$ 直接特殊处理；否则先枚举 R' 的子集 R ，这里限制 $p_R < p_{R'}$ 。把 $(s_{R-R'}, dp_{LR})$ 这个 pair 从小到大排序，并处理出前缀的信息和。

再枚举 L 的超集 L' ，找到第一个值 $\leq W - s_{L'-L}$ 的前缀，用相应的信息更新 $dp_{L'R}$ 即可。

细节：这里不需要先枚举 L, R' 再排序。可以在 dp 的进程开始前枚举 R' ，预处理出 $s_{R-R'}$ 排序的结果；同理枚举 L 预处理。时间复杂度 $O(6^{\frac{n}{2}} + 3^{\frac{n}{2}}n)$ 。

致谢

谢谢大家聆听